

Zero to Hero Study Handbook: Production-Grade Multimodal AI Platform

This handbook is based on static analysis of this repository only. It is grounded in the real source files under `src/multimodal_ai`, root launchers, `configs/`, and supporting scripts/tests.

Module 1: Foundations & Architecture

1.1 What this project does

This project is a local-first multimodal AI platform that exposes one shared service layer through four interfaces:

- FastAPI HTTP API (`src/multimodal_ai/api/app.py`)
- Typer CLI (`src/multimodal_ai/cli/main.py`)
- Streamlit UI (`src/multimodal_ai/ui/streamlit_app.py`)
- MCP tool server (`src/multimodal_ai/mcp/server.py`)

Main use cases implemented in PlatformService (`src/multimodal_ai/services/platform_service.py`):

- Caption generation (`caption`)
- OCR and document text extraction (`ocr`)
- Embedding generation (`embeddings`)
- Semantic search and retrieval (`search`, `retrieve`)
- Visual question answering (`vqa`)
- Image comparison (`compare`)
- Combined multimodal analysis (`analyze`)
- Document ingest for RAG (`documents`)
- Runtime analytics (`analytics`)

1.2 Core paradigms and patterns used in this codebase

Definitions first:

- Adapter pattern: Wraps model/provider-specific logic behind common interfaces.
- Registry pattern: Stores adapter factories by string key and creates instances at runtime.
- Pipeline pattern: Breaks processing into stages (extract, chunk, embed, retrieve, answer).
- Service orchestration pattern: One top-level service coordinates all pipelines and stores.
- Repository/storage wrapper pattern: Dedicated classes isolate SQLite and Chroma operations.
- Graceful fallback pattern: Adapters return deterministic fallback outputs when model backends are unavailable.

Where these appear:

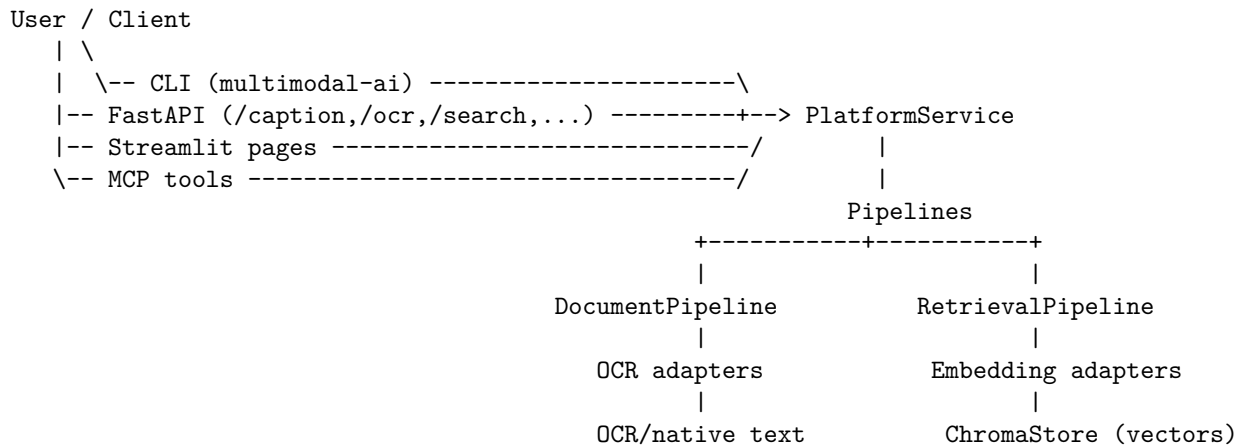
- Adapter interfaces: `src/multimodal_ai/adapters/base.py`
- Adapter registry: `src/multimodal_ai/adapters/registry.py`
- Built-in adapter registration: `src/multimodal_ai/services/bootstrap.py` (`build_registry`)
- Pipelines: `src/multimodal_ai/pipelines/document_pipeline.py`, `retrieval_pipeline.py`, `rag_pipeline.py`
- Service orchestrator: `src/multimodal_ai/services/platform_service.py`
- Storage wrappers: `src/multimodal_ai/storage/sqlite_store.py`, `chroma_store.py`
- Fallback behavior:
- Vision fallback class `_FallbackVisionAdapter` in `src/multimodal_ai/adapters/vision.py`
- Embedding fallback via `deterministic_vector` in `src/multimodal_ai/adapters/common.py`
- LLM fallback response in `OllamaLLMAdapter.complete` and `HFTTextGenerationAdapter.complete` in `src/multimodal_ai/adapters/llm.py`

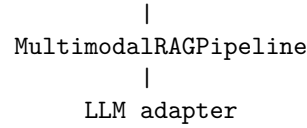
1.3 Architecture components and interaction

High-level component graph:

- Interface layer: API, CLI, UI, MCP
- Core orchestrator: `PlatformService`
- Pipelines: document, retrieval, multimodal RAG
- Plugin/adapters: vision, OCR, embedding, detection, LLM
- Persistence:
- SQLite (`data/multimodal.db`) for metadata/history/usage
- Chroma (`data/chroma`) for vectors
- Observability:
- In-memory latency metrics (`MetricsCollector`)
- System telemetry (`collect_system_stats`)
- Structured logging (`observability/logging.py`)

ASCII main flow:





PlatformService also persists to SQLiteStore:
 assets, captions, ocr_results, detections, processing_history, model_usage_metrics

Module 2: Repository Map

The table below focuses on files you should learn first.

File/Directory Path	Primary Responsibility
pyproject.toml	Project metadata, dependencies, tool config, CL
app.py	FastAPI launcher shim
streamlit_app.py	Streamlit launcher shim
mcp_server.py	MCP launcher shim
configs/config.yaml	Root Hydra/OmegaConf tree
configs/models/default.yaml	Model defaults and supported model names
configs/ocr/default.yaml	OCR policy
configs/retrieval/default.yaml	Retrieval hyperparameters
configs/runtime/default.yaml	Runtime defaults
configs/prompts/default.yaml	Prompt objective/constraints/output schemas
src/multimodal_ai/domain.py	Shared request/response contracts
src/multimodal_ai/config/settings.py	Typed config loading from YAML
src/multimodal_ai/services/bootstrap.py	Dependency graph builder
src/multimodal_ai/services/platform_service.py	Core orchestration logic
src/multimodal_ai/adapters/base.py	Adapter interfaces/contracts
src/multimodal_ai/adapters/registry.py	Runtime adapter factory registry
src/multimodal_ai/adapters/vision.py	Vision adapter implementations and fallback
src/multimodal_ai/adapters/embedding.py	Embedding adapter implementations
src/multimodal_ai/adapters/ocr.py	OCR adapter implementations
src/multimodal_ai/adapters/detection.py	Detection adapters
src/multimodal_ai/adapters/llm.py	LLM adapters with fallback behavior
src/multimodal_ai/pipelines/document_pipeline.py	Native parse + OCR routing
src/multimodal_ai/pipelines/retrieval_pipeline.py	Indexing and semantic retrieval
src/multimodal_ai/pipelines/rag_pipeline.py	RAG ingest and answer workflow
src/multimodal_ai/storage/sqlite_models.py	ORM schema
src/multimodal_ai/storage/sqlite_store.py	SQLite operations
src/multimodal_ai/storage/chroma_store.py	Chroma vector operations
src/multimodal_ai/api/app.py	FastAPI app and routes
src/multimodal_ai/api/schemas.py	API request schema
src/multimodal_ai/api/dependencies.py	Service dependency provider
src/multimodal_ai/cli/main.py	CLI commands
src/multimodal_ai/ui/streamlit_app.py	Multi-page Streamlit dashboard

File/Directory Path	Primary Responsibility
src/multimodal_ai/mcp/server.py	MCP tool server bindings
src/multimodal_ai/mcp/external.py	External MCP hook registry
src/multimodal_ai/analytics/metrics.py	In-memory latency aggregation
src/multimodal_ai/analytics/system_monitor.py	CPU/GPU telemetry probes
src/multimodal_ai/observability/logging.py	Rich logger setup
tests/integration/test_api_contracts.py	Route/OpenAPI contract checks
tests/unit/*.py	Unit checks for domain, registry, CLI help, doc p

Module 3: Core Execution Flows

This module traces the main runtime paths with exact symbols from the code.

3.1 Canonical request and response envelopes

Defined in `src/multimodal_ai/domain.py`.

Input payload shape (InputPayload):

```
class InputPayload(BaseModel):
    text: str | None = None
    question: str | None = None
    image_path: str | None = None
    image_paths: list[str] = Field(default_factory=list)
    document_path: str | None = None
    query: str | None = None
```

Standard request envelope (RequestEnvelope):

- input: InputPayload
- options: dict[str, Any]
- model_overrides: dict[str, str]
- trace: TraceContext (trace_id, source)

Standard response envelope (ResponseEnvelope):

- status: "ok" | "error"
- result: dict[str, Any]
- confidence: float | None
- latency_ms: float
- artifacts: dict[str, Any]
- trace_id: str
- errors: list[ErrorInfo]
- timestamp: datetime

3.2 Flow A: FastAPI request to service response

Files:

- `src/multimodal_ai/api/app.py`
- `src/multimodal_ai/api/schemas.py`
- `src/multimodal_ai/api/dependencies.py`
- `src/multimodal_ai/services/platform_service.py`

Step-by-step:

1. `create_app()` defines route handlers (e.g., `@app.post("/caption")`).
2. FastAPI parses JSON body into `APIRequest` (`schemas.py`).
3. `APIRequest.to_envelope()` converts it into `RequestEnvelope`.
4. Handler resolves singleton `PlatformService` via `Depends(get_platform_service)`.
5. `PlatformService` method executes business logic and returns `ResponseEnvelope`.
6. Handler returns `model_dump()` to HTTP response JSON.

Route set implemented in `api/app.py`:

- GET `/health`
- POST `/caption`
- POST `/search`
- POST `/retrieve`
- POST `/vqa`
- POST `/ocr`
- POST `/compare`
- POST `/analyze`
- POST `/documents`
- POST `/embeddings`
- POST `/analytics`

3.3 Flow B: Caption generation (`/caption` and CLI `caption`)

Method: `PlatformService.caption`.

Execution path:

1. Read `image_path` from `request.input.image_path`.
2. Resolve `style` from `request.options["style"]` (default `detailed`).
3. Resolve vision model name:
 - `request.model_overrides["vision"]` if present
 - else `PlatformConfig.default_vision_model`
4. Instantiate vision adapter with `self._registry.create_vision(model_name)`.
5. Call `vision.caption(image_path=image_path, style=style)`.
6. Persist metadata in SQLite:
 - `add_asset(...)`
 - `add_caption(...)`
7. Record metrics/history with `_record("caption", ...)`.
8. Return `ResponseEnvelope` with payload from adapter.

Caption result shape (from adapter contract in `vision.py`):

```
{
  "model": "qwen2_5_vl or ollama:<model>",
  "style": "short|detailed|social|technical|alt_text",
  "caption": "string",
  "confidence": 0.35
}
```

3.4 Flow C: OCR and document extraction (/ocr)

Primary code:

- `PlatformService.ocr`
- `DocumentPipeline.run`

Routing rules in `DocumentPipeline.run(file_path)`:

1. If extension is image (`.png/.jpg/.jpeg/.webp/.tiff`):
 - call configured OCR adapter (`registry.create_ocr(primary_engine).extract(...)`)
 - return `OCRResult(engine, text, blocks, tables, is_scanned)`
2. Else attempt native extraction:
 - PDF: `_extract_pdf_text` (`pdfplumber`)
 - DOCX: `_extract_docx_text` (`python-docx`)
 - PPTX: `_extract_pptx_text` (`python-pptx`)
3. If extracted native text length $\geq$ `min_text_chars`, return `engine="native_parser"`.
4. Otherwise fallback to OCR:
 - if PDF, render first page to PNG via `_render_pdf_first_page` (`fitz / PyMuPDF`)
 - run OCR adapter on rendered image or original file
 - return `OCRResult` with `is_scanned=True`

`OCRResult` output fields:

- `engine`: str
- `text`: str
- `blocks`: list[`OCRBlock`] where each block has `text`, `bbox`, `confidence`
- `tables`: list[dict[str, Any]]
- `is_scanned`: bool

3.5 Flow D: Retrieval and semantic search (/search and /retrieve)

Key symbols:

- `PlatformService.search`
- `RetrievalPipeline.search`
- `ChromaStore.search`

Execution path:

1. Query source:
 - `request.input.query` or fallback `request.input.text`
2. Modality:
 - `request.options["modality"]` default "image"
3. `top_k`:
 - `request.options["top_k"]` or `PlatformConfig.retrieval_top_k`
4. `RetrievalPipeline.search` embeds query text using selected embedding adapter.
5. `ChromaStore.search` queries collection mapped by modality:
 - "image" -> "image_embeddings"
 - "document" -> "document_embeddings"
 - "ocr" -> "ocr_embeddings"
 - "screenshot" -> "screenshot_embeddings"
 - "chart" -> "chart_embeddings"
6. Distances are converted to score: `score = 1/(1+distance)`.
7. Result returned as `RetrievalHit` list with `id`, `score`, `text`, `metadata`.

Exact payload returned by `PlatformService.search`:

```
{
  "query": "string",
  "modality": "image|document|ocr|screenshot|chart",
  "top_k": 5,
  "hits": [
    {
      "id": "string",
      "score": 0.0,
      "text": "optional string",
      "metadata": {}
    }
  ]
}
```

`PlatformService.retrieve` is an alias that directly calls `self.search(request)`.

3.6 Flow E: Multimodal RAG (/documents, document vqa)

Key symbols:

- `MultimodalRAGPipeline.ingest_document`
- `MultimodalRAGPipeline.answer`
- `PlatformService.documents`

- PlatformService.vqa

Ingest flow (/documents):

1. DocumentPipeline.run(path) gets OCR/native text.
2. _chunk(text) slices text using config-based chunk_size and chunk_overlap.
3. Each chunk indexed by RetrievalPipeline.index_text(..., modality="document", metadata={"source": path}).
4. Returns:
 - ocr (full OCRResult dump)
 - chunks_indexed: int
 - chunk_ids: list[str]

Question answering flow (/vqa with document_path):

1. PlatformService.vqa sees no image_path; routes to RAG answer path.
2. MultimodalRAGPipeline.answer(question, path, top_k=5) optionally ingests document first.
3. Retrieves top document chunks via semantic search.
4. Builds context string from hit texts.
5. Creates LLM via registry.create_llm(self._llm_name).
6. Sends prompt to llm.complete(prompt).
7. Returns:
 - answer
 - confidence (0.7 if hits else 0.2)
 - retrieval_hits

3.7 Flow F: Image comparison (/compare)

Key symbols:

- PlatformService.compare
- _cosine_similarity(vec_a, vec_b)

Execution path:

1. Input images from request.input.image_paths.
2. Guard: if fewer than 2, returns payload error Need at least two images.
3. Embed each image using selected embedding model.
4. Compare each image against first image with cosine similarity.
5. Build scores list with pair and similarity.
6. Compute avg_similarity and derive:
 - summary: "Images are visually close" if > 0.7 else "Images differ materially"
 - quality_assessment: "high" if > 0.8 else "moderate"

Output payload shape:


```

{
  "scores": [
    {
      "pair": ["pathA", "pathB"],
      "similarity": 0.0
    }
  ],
  "summary": "Images are visually close",
  "quality_assessment": "high|moderate"
}

```

3.8 Flow G: Comprehensive analysis (/analyze)

Method: PlatformService.analyze.

If image_path exists:

- runs vision.caption(..., style="detailed")
- runs vision.caption(..., style="technical")
- runs detector.detect(image_path) using "yolo" detector
- persists detections to SQLite (add_detections)

If document_path exists:

- runs DocumentPipeline.run(doc_path)
- runs RAG answer for summary (question fallback: "Summarize key insights")

Returns combined payload with available sections:

- caption
- technical_description
- detections
- document
- document_summary

3.9 Flow H: Interface-specific orchestration

CLI (src/multimodal_ai/cli/main.py):

- Each command creates RequestEnvelope with TraceContext(source="cli")
- Calls matching PlatformService method
- Prints JSON via Rich (console.print_json)

Streamlit (src/multimodal_ai/ui/streamlit_app.py):

- Uses @st.cache_resource service singleton
- Sidebar routes to pages:
- Dashboard, Image Upload, Caption Generator, OCR, Semantic Search, Image Similarity, VQA, Chart Analyzer, Document Analyzer, Analytics, Settings

- Image upload page saves file to `data/uploads` and calls `service.index_asset(...)`

MCP (`src/multimodal_ai/mcp/server.py`):

- `build_mcp_server()` initializes `FastMCP("multimodal-ai-platform")`
- Exposes tools: `ocr`, `caption`, `search`, `vqa`, `embeddings`, `retrieve`
- Each tool wraps a `PlatformService` call and returns `json.dumps(response.model_dump(mode="json"))`

Module 4: Setup & Run Guide

4.1 Prerequisites

From `code/manifests`:

- Python 3.12.x (`pyproject.toml: requires-python ==3.12.*`)
- `uv` package manager
- Linux/Unix shell environment

External runtimes used by adapters:

- Ollama HTTP API at `http://localhost:11434` for:
- `OllamaLLMAdapter`
- `OllamaVisionAdapter`
- `GLMOcrAdapter`
- Optional local model assets for Ultralytics YOLO (`yolov8n.pt` exists in repo root)

4.2 Install dependencies on a clean machine

```
uv venv .venv
source .venv/bin/activate
uv sync --extra dev
```

Optional quality tools (declared in dev deps):

- `pytest`, `ruff`, `mypy`, `playwright`, `wheel`

4.3 Configuration files to understand first

Primary config load path:

- `configs/config.yaml` (loaded by `load_config(path="configs/config.yaml")`)

Merged config groups:

- `models: default -> configs/models/default.yaml`
- `ocr: default -> configs/ocr/default.yaml`
- `retrieval: default -> configs/retrieval/default.yaml`
- `runtime: default -> configs/runtime/default.yaml`
- `prompts: default -> configs/prompts/default.yaml`

High-impact keys:

- `storage.sqlite_url`
- `storage.chroma_path`
- `models.vision.default`
- `models.llm.backend`
- `models.llm.default_model`
- `ocr.primary_engine`
- `retrieval.top_k`
- `retrieval.chunk_size`
- `retrieval.chunk_overlap`
- `runtime.allow_network_download`

4.4 Environment variables and .env keys

Required `.env` keys in current codebase:

- None are mandatory for startup.

Optional env var used in code:

- `MM_USE_OLLAMA_VISION`
- Location: `src/multimodal_ai/services/bootstrap.py`
- Effect:
 - if `true`, registers vision backends as Ollama-based adapters for `qwen2_5_vl` and `llama_vision`
 - if `false` (default), registers fallback vision adapters for those names

4.5 Main startup commands (entry points)

FastAPI:

```
uv run uvicorn app:app --host 0.0.0.0 --port 8000
```

Streamlit:

```
uv run streamlit run src/multimodal_ai/ui/streamlit_app.py --server.address 0.0.0.0 --server
```

CLI:

```
uv run multimodal-ai --help
uv run multimodal-ai doctor
uv run multimodal-ai caption data/uploads/example.png --style detailed
```

MCP server:

```
uv run python mcp_server.py
```

4.6 Database migration/seeding notes

SQLite:

- No Alembic migration command is wired into runtime startup.
- Table creation is automatic in `build_platform_service()`:

- `sqlite_store.create_tables()` executes `Base.metadata.create_all`.

Chroma:

- Collections are auto-created in `ChromaStore.__init__` via `get_or_create_collection`.

Seeding/indexing:

- No standalone seeding script is required by code.
- Data gets indexed when:
 - calling `PlatformService.index_asset(...)` (used by Streamlit "Image Upload" page)
 - calling `/documents` (RAG ingest path)
 - calling `analyze/caption/ocr` paths that persist metadata

Module 5: Study Plan & Practice Exercises

5.1 Ordered study plan

1. Read contracts first:
 - `src/multimodal_ai/domain.py`
 - `src/multimodal_ai/config/settings.py`
2. Learn extension architecture:
 - `src/multimodal_ai/adapters/base.py`
 - `src/multimodal_ai/adapters/registry.py`
 - then concrete adapters in `adapters/*.py`
3. Understand persistence model:
 - `src/multimodal_ai/storage/sqlite_models.py`
 - `src/multimodal_ai/storage/sqlite_store.py`
 - `src/multimodal_ai/storage/chroma_store.py`
4. Understand data-processing pipelines:
 - `src/multimodal_ai/pipelines/document_pipeline.py`
 - `src/multimodal_ai/pipelines/retrieval_pipeline.py`
 - `src/multimodal_ai/pipelines/rag_pipeline.py`
5. Read orchestrator:
 - `src/multimodal_ai/services/platform_service.py`
 - `src/multimodal_ai/services/bootstrap.py`
6. Read interface layers:
 - `src/multimodal_ai/api/app.py`
 - `src/multimodal_ai/cli/main.py`
 - `src/multimodal_ai/ui/streamlit_app.py`
 - `src/multimodal_ai/mcp/server.py`

7. Confirm assumptions with tests:
 - `tests/unit/*.py`
 - `tests/integration/test_api_contracts.py`
8. Finally review automation scripts and docs:
 - `scripts/*.py`
 - `README.md`
 - `docs/ARCHITECTURE.md`

5.2 Practice exercises

1. Exercise: Trace `/caption` end-to-end.
 - Task: Starting at `api/app.py`, list every function called until DB persistence.
2. Exercise: Explain dynamic model switching.
 - Task: Identify exactly where `model_overrides` can replace config defaults.
3. Exercise: Reconstruct OCR routing logic.
 - Task: Describe all branches in `DocumentPipeline.run` for image, PDF, DOCX, PPTX, and fallback OCR.
4. Exercise: Add a new embedding adapter key mentally.
 - Task: Show which files/methods must change to register a new embedding adapter named `my_embed`.
5. Exercise: Explain search scoring math.
 - Task: Derive how Chroma distance is transformed into similarity score.
6. Exercise: Compare `/search` and `/retrieve`.
 - Task: Prove whether they are different or aliases.
7. Exercise: Identify persisted artifacts for `analyze`.
 - Task: Which SQLite tables are written when `analyze` is called with image only?
8. Exercise: Map Streamlit pages to service methods.
 - Task: For each page in sidebar, identify the service call(s) used.
9. Exercise: Understand MCP tool wrapping.
 - Task: Explain why MCP tools return `str` and where JSON serialization happens.
10. Exercise: Explain startup side effects.
 - Task: List which resources are initialized by `build_platform_service()`.

5.3 Solution outlines

1. /caption flow outline:

- `api/app.py` route -> `APIRequest.to_envelope()` -> `PlatformService.caption()`
-> `registry.create_vision()` -> `vision.caption()` -> `sqlite.add_asset()`
-> `sqlite.add_caption()` -> `_record()` -> response.

2. Model override outline:

- In `PlatformService.caption`, `vqa` (image branch), `embeddings`, `compare`, `search` (indirect via default embedding), and `analyze`, model names are read from `request.model_overrides` first, then config defaults.

3. OCR routing outline:

- Image extension: direct OCR adapter.
- Doc extension: native parser first.
- Native text above threshold: `engine="native_parser"`.
- Else fallback OCR; PDF may render first page to PNG before OCR.

4. New embedding adapter outline:

- Implement adapter in `adapters/embedding.py` (or new file) subclassing `EmbeddingAdapter`.
- Register in `build_registry()` using `registry.register_embedding("my_embed", Factory)`.
- Use by setting config `models.embeddings.vision` or request `model_overrides["embedding"]`.

5. Search scoring outline:

- `ChromaStore.search` reads `distance`; score is `1 / (1 + distance)`.

6. /search vs /retrieve outline:

- `PlatformService.retrieve` returns `self.search(request)` with no additional logic.

7. analyze image-only persistence outline:

- Writes `assets` row through `add_asset`.
- Writes `detections` rows through `add_detections` if detections exist.
- Records `processing_history` event through `_record`.
- May update `model_usage_metrics` for action "analyze" and selected model.

8. Streamlit page mapping outline:

- Dashboard -> `service.health()`
- Image Upload -> `service.index_asset(...)`
- Caption Generator -> `service.caption(...)`
- OCR -> `service.ocr(...)`

- Semantic Search -> `service.search(...)`
- Image Similarity -> `service.compare(...)`
- VQA -> `service.vqa(...)`
- Chart Analyzer -> `service.analyze(...)`
- Document Analyzer -> `service.analyze(...)`
- Analytics -> `service.analytics()`
- Settings -> reads service config fields.

9. MCP JSON string outline:

- In `mcp/server.py`, each `@mcp.tool()` function returns `json.dumps(service.<method>(...).model_dump())` so tool output type is `str`.

10. Startup side effects outline:

- Load config via `load_config`.
- Build and register adapters.
- Initialize SQLite engine and create tables.
- Initialize persistent Chroma client and collections.
- Build Document/Retrieval/RAG pipelines.
- Create metrics collector and external MCP hook registry.
- Return a ready `PlatformService` instance.

Learner Verification Checklist

Use this checklist before moving to implementation work:

- I can explain the role of `RequestEnvelope` and `ResponseEnvelope` and list their fields from memory.
- I can trace one full API route (for example `/caption`) from FastAPI handler to adapter call to persistence.
- I can explain how `DocumentPipeline.run` decides native parsing vs OCR fallback.
- I can explain how chunking works in `MultimodalRAGPipeline._chunk` and why overlap exists.
- I can describe how adapter registration in `build_registry` enables runtime model switching.
- I can list the Chroma collections and what modality each one stores.
- I can list the SQLite tables and what each table represents.
- I can explain the difference between Streamlit/UI orchestration and API orchestration in this codebase.
- I can explain where fallback behavior is implemented for vision, embedding, and LLM adapters.
- I can point to the exact optional environment variable and describe its runtime effect.